

AI Multi-Object Detection System

Project README — YOLOv8 / OpenCV / PyTorch

An AI-powered multi-object detection system built using YOLOv8, OpenCV, and PyTorch. This project was developed as part of an internship project at Hindalco Industries Limited, and detects and tracks multiple objects in real time from images, videos, or live camera feeds.

Note: Descriptions for `main.py`, `dd.py`, `tt.py`, and `convert_xml_to_yolo.py` are inferred from standard YOLOv8 project conventions and typical file naming, since only the file/folder structure (not code content) was available while writing this document. Please verify against your actual code and edit as needed.

Project Structure

```
AI MULTI OBJECT DETECTION/
|
|-- .venv/                                # Python virtual environment (auto-generated)
|   |-- Include/
|   |-- Lib/
|   |-- Scripts/
|   |-- .gitignore
|   `-- pyvenv.cfg
|
|-- dataset/                             # Training/validation/testing data
|   |-- test/                            # Test images + labels
|   |-- train/                           # Training images + labels
|   |-- valid/                           # Validation images + labels
|   `-- data.yaml                        # Dataset configuration file (classes, paths)
|
|-- outputs/                             # Saved output files (images/videos)
|-- runs/                                # YOLO auto training/inference logs
|-- venv/                                # (Possible duplicate) venv folder
|-- videos/                              # Input videos for testing/inference
|
|-- .gitattributes                        # Git config for file types (LFS etc.)
|-- convert_xml_to_yolo.py               # Converts XML annotations to YOLO .txt format
|-- dd.py                               # Custom detection/data-processing script
|-- main.py                             # Main entry point - runs the detection pipeline
|-- output.avi                           # Sample output video (AVI) from a detection run
|-- output.mp4                           # Sample output video (MP4) from a detection run
|-- requirements.txt                     # Python dependencies needed to run the project
|-- tt.py                                # Test/training helper script
|
|-- yolo26n.pt                           # Trained/pretrained YOLO model weight file
|-- yolov8m.pt                           # YOLOv8 'medium' weight file
|-- yolov8n.pt                           # YOLOv8 'nano' weight file (fastest)
|-- yolov8s.pt                           # YOLOv8 'small' weight file
```

File-by-File Explanation

`main.py`

This is the main script of the project — the entry point you run to start object detection. It typically:

- Loads a trained YOLO model (.pt file)
- Loads an input source (image, video, or webcam)
- Runs inference (detection) frame-by-frame
- Draws bounding boxes + labels on detected objects
- Saves or displays the output

Run it with:

```
python main.py
```

[convert_xml_to_yolo.py](#)

A data preparation utility. If your dataset was originally annotated in XML format (e.g., using tools like Labellmg in Pascal VOC format), this script converts those XML annotation files into YOLO .txt label format (class_id x_center y_center width height, normalized between 0–1), which is what YOLOv8 requires for training.

You'd use this script once, right after collecting/annotating a new dataset in XML, before training.

[dd.py](#)

Appears to be a helper/detection script — possibly used for testing detection on a specific image/video, debugging model outputs, or a draft/duplicate version of the detection logic. Recommend renaming this to something descriptive (e.g. detect_debug.py) for clarity, or removing it if unused.

[tt.py](#)

Likely a training or testing script — possibly used to train the YOLO model on your dataset (model.train(...)) or to run quick test inferences. Recommend a clearer filename such as train.py or test.py.

[requirements.txt](#)

Lists all the Python packages this project depends on (e.g. ultralytics, opencv-python, torch, numpy, etc.). Install them all at once with:

```
pip install -r requirements.txt
```

[output.avi / output.mp4](#)

Sample output videos generated after running detection on an input video — showing bounding boxes and labels drawn on detected objects. These are results, not code — safe to delete/regenerate anytime.

[.gitattributes](#)

A Git configuration file that tells Git how to handle certain file types (e.g., normalize line endings, treat .pt model files as binary/LFS). Doesn't affect how the code runs — only version control behavior.

[Model weight files \(.pt\)](#)

These are trained YOLO model files (PyTorch format):

File	Meaning
------	---------

yolov8n.pt	YOLOv8 Nano — smallest & fastest, lower accuracy
yolov8s.pt	YOLOv8 Small — balance of speed & accuracy
yolov8m.pt	YOLOv8 Medium — more accurate, slower
yolo26n.pt	Likely a custom-trained model (or newer YOLO version — verify naming)

main.py loads one of these to perform detection. You can swap between them depending on your speed vs. accuracy needs.

The dataset/ Folder

```
dataset/
|-- train/      -> Images + labels used to train the model
|-- valid/      -> Images + labels used to validate the model during training
|-- test/       -> Images + labels used to test the final model
`-- data.yaml   -> Tells YOLO where the above folders are, and what classes exist
```

Each of train/, valid/, and test/ usually contains two subfolders internally:

```
train/
|-- images/     # .jpg / .png files
`-- labels/     # .txt files (one per image, YOLO format annotations)
```

Example data.yaml

```
train: dataset/train/images
val: dataset/valid/images
test: dataset/test/images

nc: 3                # number of classes
names: ['person', 'car', 'bike'] # class names, in order matching class IDs
```

How to Change / Replace the Dataset

Follow these steps if you want to train the model on a new or different dataset:

Step 1: Collect & Annotate New Images

- Gather your new images.
- Annotate each object with a bounding box using a tool like LabelImg, Roboflow, or CVAT.
- Export annotations in YOLO format directly if possible. If you only have XML (Pascal VOC) annotations, continue to Step 2.

Step 2: Convert Annotations (if needed)

If your new annotations are in XML format:

```
python convert_xml_to_yolo.py
```

This generates .txt label files in YOLO format for each image. Check inside the script for the exact input/output folder paths it expects — update them if your new data is in a different location.

Step 3: Organize Folder Structure

Replace the contents of the dataset/ folder with your new data, following this exact structure:

```
dataset/
|-- train/
|   |-- images/    <- ~70-80% of your new images
|   |-- labels/    <- matching YOLO .txt files
|-- valid/
|   |-- images/    <- ~10-15% of images, for validation
|   |-- labels/
|-- test/
|   |-- images/    <- ~10-15% of images, for final testing
|   |-- labels/
`-- data.yaml
```

Each image file must have a matching .txt label file with the same filename (e.g. img001.jpg <-> img001.txt).

Step 4: Update data.yaml

Edit dataset/data.yaml to reflect:

- The correct folder paths (if changed)
- The number of classes (nc)
- The list of class names, in the exact order matching the class IDs used in your labels

Example — if your new dataset has 5 classes instead of 3:

```
train: dataset/train/images
val: dataset/valid/images
test: dataset/test/images

nc: 5
names: ['helmet', 'vest', 'gloves', 'boots', 'goggles']
```

Step 5: Retrain the Model

Use the Ultralytics YOLO training command (in tt.py, or directly in a new script/terminal):

```
from ultralytics import YOLO

model = YOLO('yolov8n.pt') # start from a pretrained base model
model.train(data='dataset/data.yaml', epochs=100, imgsz=640)
```

Or from the command line:

```
yolo task=detect mode=train model=yolov8n.pt data=dataset/data.yaml epochs=100 imgsz=640
```

This creates a new folder inside runs/detect/train/ containing:

- weights/best.pt -> your newly trained model (use this in main.py going forward)
- weights/last.pt -> the last checkpoint
- Training graphs, confusion matrix, sample predictions, etc.

Step 6: Update main.py to Use the New Model

In main.py, change the model path from an old .pt file to your newly trained one:

```
model = YOLO('runs/detect/train/weights/best.pt')
```

Step 7: Run Detection

```
python main.py
```

Installation & Setup

- 1. Clone/download the project.
- 2. Create and activate a virtual environment (see below).
- 3. Install dependencies with `pip install -r requirements.txt`
- 4. Run the detection system with `python main.py`

```
python -m venv .venv
.venv\Scripts\activate      # Windows
source .venv/bin/activate   # macOS/Linux

pip install -r requirements.txt
python main.py
```

Housekeeping Notes

- You currently have two virtual environment folders (`.venv/` and `venv/`) — consider removing one to avoid confusion and save space.
- `__pycache__/` contains compiled Python bytecode (`.pyc` files) — safe to delete anytime, Python regenerates it automatically.
- Consider adding a `.gitignore` to exclude `.venv/`, `venv/`, `__pycache__/`, `runs/`, and `outputs/` from version control, since these are large/auto-generated.

Tech Stack

- Python
- YOLOv8 (Ultralytics)
- OpenCV — video/image processing
- PyTorch — deep learning backend

README generated based on project file structure. Please review script descriptions (`main.py`, `dd.py`, `tt.py`, `convert_xml_to_yolo.py`) against your actual code and adjust any details that don't match.